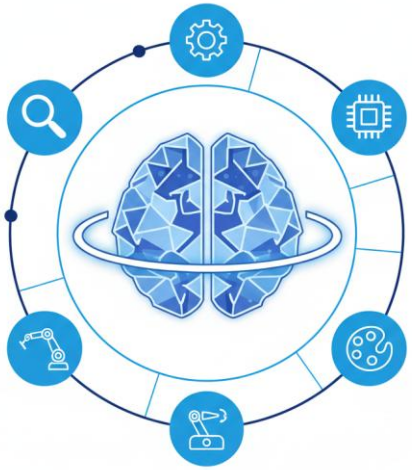
파이썬 객체와 모듈

함수/ 클래스/ 모듈



함수(function)

+ 파이썬의 함수에 대해 알려줘

1️⃣ 함수란?

함수는 **특정 작업을 수행하는 코드 묶음**이에요.

👉 한 번 만들어 두면

- 여러 번 사용할 수 있고
- 코드가 깔끔해지고
- 수정도 쉬워져요

예: "두 수를 더하는 기능", "이름을 출력하는 기능" 같은 것들

함수(function)

- **사용자 정의 함수의 형태**

```
def 함수 이름():
    함수의 내용
```

4칸 들여쓰기
인덴트(indent)

return 반환값이 없는 함수

```
def 함수 이름(매개 변수):
    함수의 내용
    return 반환값
```

return 반환값이 있는 함수

함수(function)

- **return이 없는 함수**

```
def greet():
    print("Hello, World!")

def greet2(name):
    print(f"Hello, {name}!")

greet()  # Hello, World!
greet2("Elsa")  # Hello, Elsa!
greet2("명제")  # Hello, 명제!
```

함수의 정의

함수의 호출

함수(function)

- **return이 있는 함수**

```
def message():
    return "Good Luck!"

def square(x):
    return x * x

def add(a, b):
    return a + b

print("5의 제곱:", square(5))  # 5의 제곱: 25
result = add(3, 5)
print("3 + 5 =", result)  # 3 + 5 = 8
```

함수(function)

- **기본 매개변수**

  매개변수를 초기화하여 선언하고 함수 호출시 매개변수를 생략하면 기본 값으로 출력된다.

def 함수 이름**(변수1, 변수2=1):**
    코드블럭

```
# 버스 요금 계산 함수
def take_bus(fare=1200):
    print(f"버스 요금은 {fare}원입니다.")

take_bus()  # 버스 요금은 1200원입니다.
take_bus(1500)  # 버스 요금은 1500원입니다.

# 거듭제곱 계산 함수
def power(base, exponent=2):
    return base ** exponent

print("3의 제곱:", power(3))  # 3의 제곱: 9
print("2의 세제곱:", power(2, 3))  # 2의 세제곱: 8
```

함수(function)

❖ 내장 함수(Built in Function)

함수	설명	사용 예
sum(iterable)	리스트나 튜플의 모든 요소의 합을 반환	sum([1, 2, 3]) 6 sum((1.2.3)) 6
max(iterable)	리스트나 튜플의 최대값을 반환	max([1, 2, 3]) 3 max((1.2.3)) 3
round(n, digit)	숫자를 입력받아 반올림하여 돌려줌	round(4.6) 5 round(4.4) 4
eval(expression)	문자열 표현식을 숫자로 변환	eval('1+2') 3
list(s)	반복가능한 문자열을 입력받아 리스트로 반환	list("python") ['p', 'y', 't', 'h', 'o', 'n']

함수(function)

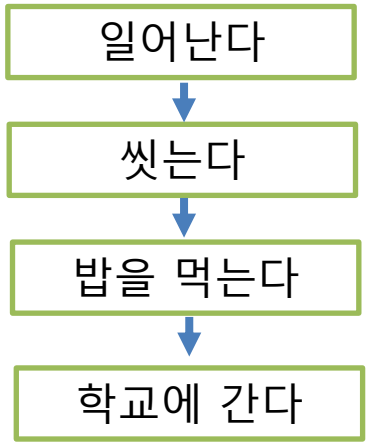
❖ 내장 함수(Built in Function)

```
a = [1, 2, 3, 4]
b = (1, 2, 3, 4)

print(sum(a))  # 합계
print(sum(b))

print(max(a))  # 최대값
print(max(b))

print(min(a))  # 최소값
print(min(b))
```

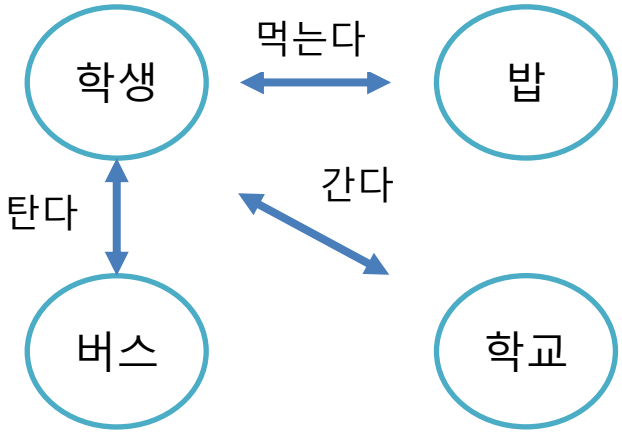
```
# 반올림
print(round(2.74))  #3
print(round(2.14))  #2

# 소수 자리수
x = 706.351
print(round(x, 1)) # 소수첫째자리, 706.4
print(round(x, 0)) # 706.0
print(round(x)) # 정수 706
print(int(round(x, -1))) # 일의 자리 710
```

클래스와 객체

- **객체(Object)란?**
  - "의사나 행위가 미치는 대상" -> 사전적 의미
  - 구체적, 추상적 데이터 단위 (구체적- 책상, 추상적-회사)
- **객체지향 프로그래밍(Objected Oriented Programming)**
  - 객체를 기반으로 하는 프로그래밍
  - 먼저 객체를 만들고 객체 사이에 일어나는 일을 구현함.



<절차지향 –C언어>



<객체지향 – Python>

클래스와 객체

- **클래스란?**
  - 객체 지향 프로그래밍(OOP)의 기본 개념 중 하나로, 관련된 속성과 메서드를 하나의 단위로 묶는 틀입니다.
  - 클래스를 사용하면 코드의 재사용성과 유지보수성을 높일 수 있습니다.
  - 클래스는 설계도와 같으며, 이를 기반으로 여러 객체(인스턴스)를 생성할 수 있습니다.
- **클래스의 속성과 기능**
  - 객체의 특성(property), 속성(attribute) -> **멤버 변수**
  - 객체가 하는 기능 -> **멤버 함수**

클래스와 객체

- 클래스 정의 및 사용

```
class 클래스 이름 :
    def __init__(self):
        멤버변수

    def 함수이름(self):
        return
```

```
객체(인스턴스) = 클래스( )

객체.멤버변수
객체.멤버 메서드
```

객체를 생성한 후 점(.) 연산자를 사용하여 멤버변수나 메서드에 접근함

클래스와 객체

- **생성자(constructor)**
  - 객체가 생성될때 자동으로 호출되는 특별한 메서드
  - 생성자는 **__init__(매개변수)**의 형태로 작성하고, 리턴값이 없다
  - 클래스 내의 모든 함수(메서드)의 매개변수에 **self**를 넣어줌

Bike	
color gears	속성(Attributes)
ride() get_info()	메서드(Methord)

클래스와 객체

- **Bike 클래스 정의 및 사용**

```python
# 클래스의 생성자(Constructor)
class Bike:
    # 생성자 메서드 - 객체가 생성될 때 자동으로 호출되는 특별한 메서드
    def __init__(self, color, gears):
        self.color = color  # 자전거 색상
        self.gears = gears  # 자전거 기어 수

    def ride(self):
        print(f"{self.color} 자전거가 {self.gears}단 기어로 달린다.")

    def get_info(self):
        return f"자전거 색상: {self.color}, 기어 수: {self.gears}"

# 클래스 사용 예시
if __name__ == "__main__":
    my_bike = Bike("blue", 21)
    print(my_bike.get_info())
    my_bike.ride()
```

모듈(Module)과 패키지

- **모듈(module)**

  모듈은 Python 파일 하나를 의미하며, 다른 Python 프로그램에서 모듈의 함수, 클래스 등을 재사용할 수 있다.

- **주요 모듈**

모 듈	설 명
math	수학 계산과 관련된 모듈
datetime	날짜 및 시간과 관련된 모듈
time	시간과 관련된 모듈
random	난수를 발생시키는 모듈
os	운영 체제(OS) 자원 제어 관련 모듈

math 모듈

- **math** 모듈

기능	함수의 사용
올림	**math.ceil(2.54)** # 3
내림	**math.floor(2.54)** # 2
제곱근	**math.sqrt(16)** # 4.0
원주율	**math.pi** # 3.1415

math 모듈

- **math 모듈**

```python
import math

# 올림
print(math.ceil(2.54))

# 반올림 - math 모듈이 아님
print(round(2.54))

# 내림(버림)
print(math.floor(2.54))

# 제곱근 - 실수로 반환
print(math.sqrt(2))
print(math.sqrt(25))
```

```
3
3
2
1.4142135623730951
5.0
3.141592653589793
원의 넓이: 50.27
```

datetime 모듈

- **datetime 모듈**

```python
import datetime

# datetime.datetime - 날짜와 시간을 사용
now = datetime.datetime.today() #오늘 날짜
print(now) # 2025-05-18 11:16:33.750807
# 년, 월, 일 출력
print(now.year)
print(now.month)
print(now.day)

# 현재 날짜 표기
print(f"{now.year}. {now.month}. {now.day}.")

# 시, 분, 초 출력
print(now.hour)
print(now.minute)
print(now.second)

# 현재 시간 표기
print(f"{now.hour} : {now.minute} : {now.second}")
```

```
2025-08-01 10:10:24.530677
2025
8
1
2025. 8. 1.
10
10
24
10 : 10 : 24
2025-08-15
2025-08-01
```

datetime 모듈

- **datetime 모듈**

```
# 특정한 날짜 설정
# datetime.date - 날짜만 사용 가능
the_day = datetime.date(2025, 8, 15)
print(the_day)

today = datetime.date.today()
print(today)

# DDay 계산
print("광복절까지 몇일 남았나(DDay)?")

remain_day = the_day - today
print(f"광복절까지 {remain_day.days}일 남았습니다.")
```

```
2025-08-15
2025-08-01
광복절까지 몇일 남았나(DDay)?
광복절까지 14일 남았습니다.
```

Random 모듈

+ 파이썬의 random 모듈을 사용하는 방법에 대해서 알려줘

1 random 모듈 불러오기

python 코드 복사

```python
import random
```

---

2 기본 난수 생성

🔹 0 이상 1 미만의 실수

python 코드 복사

```python
random.random()
```

👉 예: `0.3748293`

🔹 특정 범위의 정수

python 코드 복사

```python
random.randint(1, 10)
```

👉 **1 이상 10 이하** 정수 하나

Random 모듈

- **random 모듈**

```python
# 난수 시드 설정 - 동일한 시드를 사용하면 동일한 난수 시퀀스 생성
random.seed(42)
print(f"시드 설정 후 난수: {random.randint(1, 10)}")
random.seed(42)
print(f"시드 설정 후 난수: {random.randint(1, 10)}")
random.seed()  # 시드 초기화
print(f"시드 초기화 후 난수: {random.randint(1, 10)}")

# 동전 던지기 시뮬레이션
# '앞면' 또는 '뒷면' 중 하나를 무작위로 선택
# 방법1
coin = random.randint(0, 1)
if coin == 0:
    print("동전 던지기 결과: 앞면")
else:
    print("동전 던지기 결과: 뒷면")

# 방법2
coin = random.choice(['앞면', '뒷면'])
print(f"동전 던지기 결과: {coin}")
```

Random 모듈

```python
# 주사위 굴리기 시뮬레이션
dice_roll = random.randint(1, 6)
print(f"주사위 굴리기 결과: {dice_roll}")

# 로또 번호 생성 (1부터 45 사이의 숫자 중 6개 선택)
lotto_numbers = random.sample(range(1, 46), k=6)
print(f"생성된 로또 번호: {sorted(lotto_numbers)}")

# 리스트에서 무작위로 요소 선택
fruits = ['사과', '바나나', '딸기', '감', '사과']
selected_fruit = random.choice(fruits)
print(f"선택된 과일: {selected_fruit}")

# 리스트에서 무작위로 여러 요소 선택 (중복 허용)
selected_fruits = random.choices(fruits, k=2)
print(f"선택된 과일들 (중복 허용): {selected_fruits}")

# 리스트에서 무작위로 여러 요소 선택 (중복 비허용)
selected_fruits = random.sample(fruits, k=2)
print(f"선택된 과일들 (중복 비허용): {selected_fruits}")
```

Random 모듈

- 숫자 맞추기 게임
  - 챗GPT 프롬프트 입력

파이썬으로 숫자 맞추기 게임을 만들 거야.
1~100까지 무작위 숫자를 생성하고,
내가 숫자를 입력하면 무작위 숫자랑 비교해서 숫자가 큰지 작은지 알려줘
숫자를 맞추면 몇 회 만에 맞췄는지 알려주고 게임을 종료해

Random 모듈

- 숫자를 추측해서 맞추는 게임

```python
import random

# 1~100 사이의 무작위 숫자 생성
answer = random.randint(1, 100)
# print(answer) # 디버깅용 출력, 실제 게임에서는 제거할 것

count = 0  # 시도 횟수

print("숫자 맞추기 게임!")
print("1부터 100 사이의 숫자를 맞춰보세요.")

while True:
    guess = int(input("숫자를 입력하세요: "))
    count += 1

    if guess > answer:
        print("더 작은 숫자입니다.")
    elif guess < answer:
        print("더 큰 숫자입니다.")
    else:
        print(f"정답입니다! {count}회 만에 맞췄어요.")
        break
```

AI 사용 – 코파일럿(copilot)

❖ 생성형 AI 코파일럿

▪ 챗GPT 프롬프트 입력

\+ ai 코파일럿에 대해 설명해 줘

좋은 질문이에요 👍
AI 코파일럿(AI Copilot)은 말 그대로 *사람을 대신하는 AI*가 아니라, **사람 옆에서 함께 일해주는 똑똑한 조수**라고 보면 딱 맞아요.

🤖 AI 코파일럿이란?

AI 코파일럿은 **인공지능이 사용자의 작업을 보조**해 주는 시스템이에요.
사용자의 **의도·맥락·데이터**를 이해해서 제안, 자동완성, 분석, 요약 같은 걸 해줍니다.

> ✈️ 비유하자면
> 비행기 조종사 = 사람
> 코파일럿 = AI (옆에서 체크하고, 추천하고, 자동 처리)

AI 사용 – 코파일럿(copilot)

❖ 생성형 AI 코파일럿

▪ 챗GPT 프롬프트 입력

+ GitHub Copilot을 사용하는 방법을 알려줘

✔ **GitHub 계정**

- https://github.com ↗
- 로그인만 하면 OK

✔ **지원되는 에디터**

가장 많이 쓰는 건:

- **VS Code** ⭐ (추천)
- JetBrains (PyCharm, IntelliJ 등)
- Visual Studio

👉 여기서는 **VS Code 기준**으로 설명할게요.

✔ **GitHub Copilot 구독**

- 개인: 유료 (무료 체험 있음)
- 학생/교사: 무료
- 기업: 별도 플랜

깃허브 코파일럿(copilot)

❖ 깃허브 코파일럿

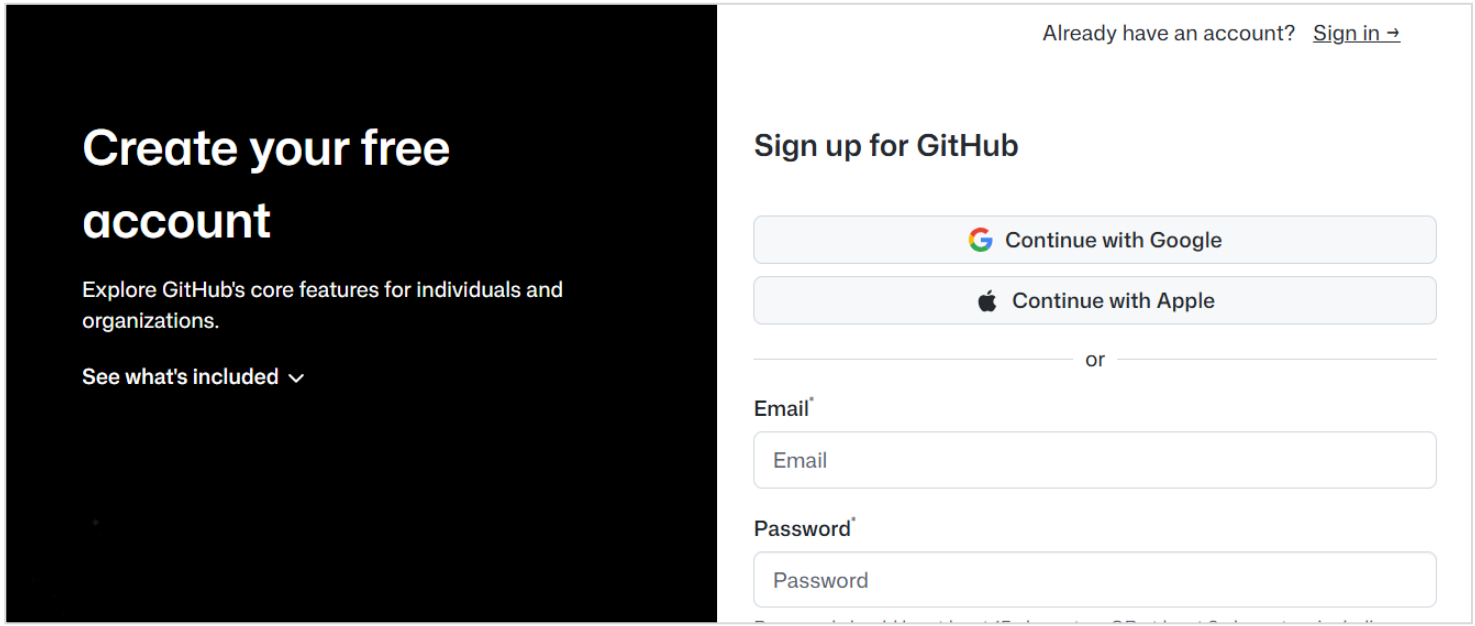
깃허브 코파일럿은 개발자가 코드를 더 빠르고 효율적으로 작성할 수 있도록 설계되었습니다. 프로그래머가 작성 중인 코드를 분석하고, 컨텍스트를 이해해 자동으로 코드를 제안합니다. 인터넷에 공개된 오픈 소스들로 학습했습니다.
깃허브(GitHub)가 출시한 프로그래밍을 돕는 인공지능입니다. 오픈 AI의 GPT 언어 모델을 이용해 깃허브 리포지터리를 학습시켜 만든 프로그램에 특화시켰습니다.

구독료 : 한 달에 10달러

AI 사용 – 코파일럿(copilot)

❖ 깃허브 계정 만들기

https://github.com/ > 회원가입(sign up)

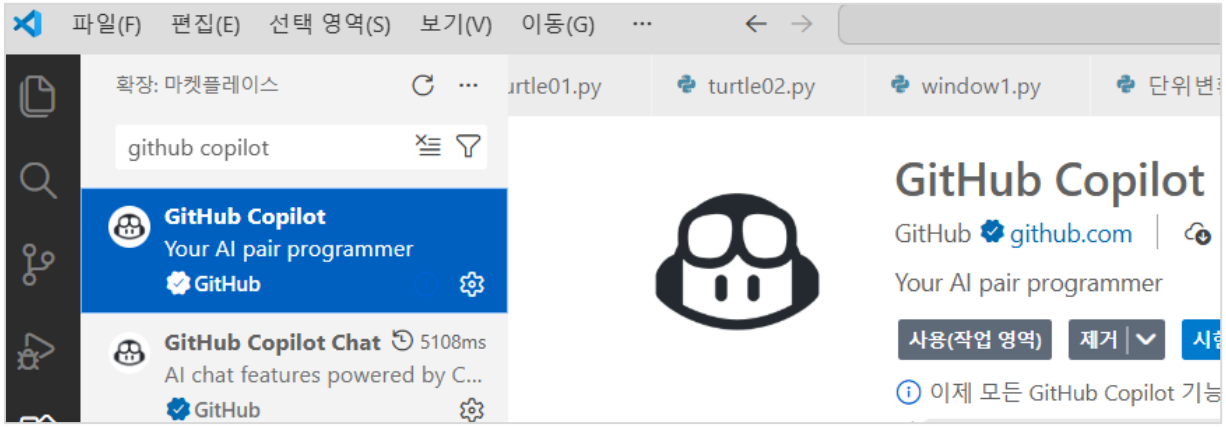


AI 사용 – 코파일럿(copilot)

❖ **VS Code에서 GitHub Copilot 사용하기**

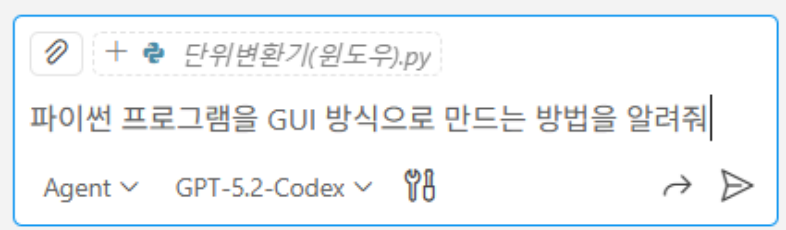
확장(Extension) > github copilot 검색 > 설치

< GitHub Copilot, GitHub Copilot Chat >



윈도우 프로그래밍

- **코파일럿 프롬프트 입력**



- **GUI 라이브러리 선택**: 표준 `tkinter`(가장 간단), `PyQt/PySide`(기능 풍부), `wxPython` 등
- **기본 흐름**: `창 생성 → 위젯 배치 → 이벤트 연결 → 실행`

예) `tkinter` 최소 구조

- `Tk()`로 창 생성
- `Label`, `Entry`, `Button` 등 위젯 추가
- `command`로 버튼 클릭 이벤트 연결
- `mainloop()`로 실행

윈도우 프로그래밍

- **GUI(Graphical User Interface)란?**

  그래픽 사용자 인터페이스를 줄여서 GUI라고 한다. GUI는 '화면'에 표시된 메뉴나 버튼으로 사용자와 상호 작용을 하는 간단한 프로그램이다.

  tkinter 라이브러리를 사용한다. -> **import tkinter**

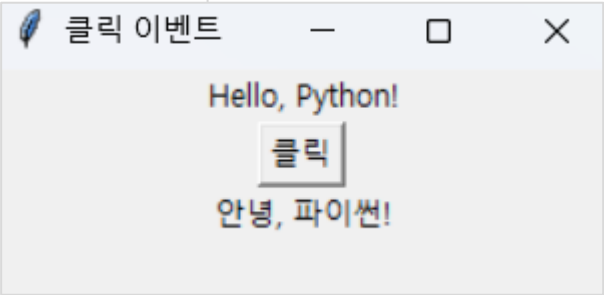
1. Tk 루트

개체이름	클래스
루트	Tk()
프레임	Frame
레이블	Label
입력상자	Entry
버튼	Button
출력상자	Text

윈도우 프로그래밍

- **클릭 이벤트 – 버튼을 눌렀을때 문자 출력**

```python
from tkinter import *

# 버튼 클릭시 호출되는 함수
def click():
    output.config(text="안녕, 파이썬!")

root = Tk() # 윈도우 창 생성
root.title("클릭 이벤트") # 창 제목 설정
root.geometry("250x100+200+100") # 가로x세로+x좌표+y좌표

# 라벨과 버튼 추가
# pack(): 위젯을 창에 추가하는 메서드
Label(root, text="Hello, Python!").pack()
Button(root, text="클릭", command=click).pack()

# 클릭후 출력 라벨
output = Label(root, text="")
output.pack()

# 이벤트 루프 시작
root.mainloop()
```



윈도우 프로그래밍

- **Button(버튼) – command**

Button(frame, text="확인", command=**click**).pack()

※ click에 괄호를 하면 함수 생성시점에서 작동하고, 괄호를 생략하면 클릭이 발생한 때 작동함

1. 콘솔에 출력

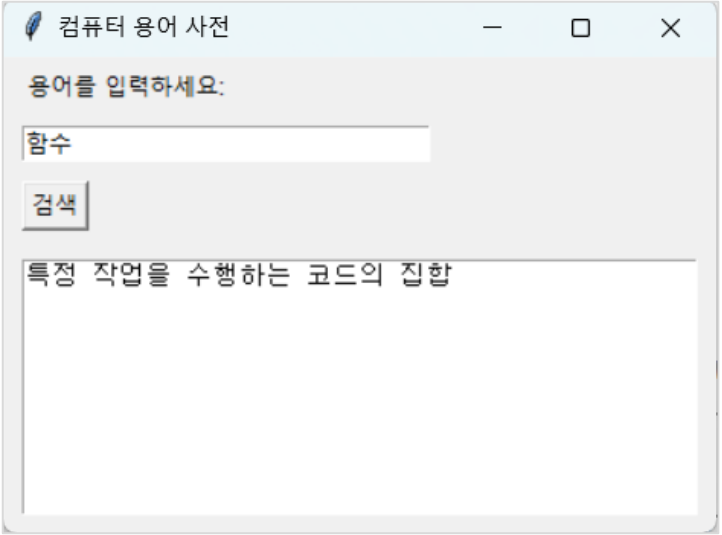
```
def click():
    print("안녕~ 파이썬!")
```

2. 레이블에 출력

```
def click():
    output.config(text="안녕, 파이썬!")
```

컴퓨터 용어사전 앱

- **컴퓨터 용어 사전 – dictionary 자료구조 이용**



App 설명

- 용어를 미리 정의한다. – 딕셔너리 자료 구조
- 단어를 입력하고 검색 버튼을 누르면, 텍스트 상자에 뜻이 출력된다.
- 검색된 단어가 없는 경우 '사전에 없는 용어입니다..'고 출력된다.

컴퓨터 용어사전 앱

- 컴퓨터 용어사전.py

```
from tkinter import *
dict = {
  "알고리즘": "문제를 해결하기 위한 절차나 방법",
  "함수": "특정 작업을 수행하는 코드의 집합",
  "변수": "데이터를 저장하는 메모리 공간의 이름",
  "클래스": "객체 지향 프로그래밍에서 객체를 생성하기 위한 틀",
}

def search():
  # 입력된 단어로 사전 검색
  try:
    word = entry.get().strip() # 입력창에서 단어 가져오기
    if not word: # 단어가 비어있으면
      output.insert(END, "용어를 입력하세요.")
      return
    meaning = dict[word] # 사전에서 뜻 찾기
    output.delete(1.0, END) # 1번째 줄, 0번째 문자 = 첫 번째 위치
    output.insert(END, meaning) # 끝 위치 (텍스트의 마지막)
  except KeyError:
    output.delete(1.0, END)
    output.insert(END, "사전에 없는 용어입니다.")
```

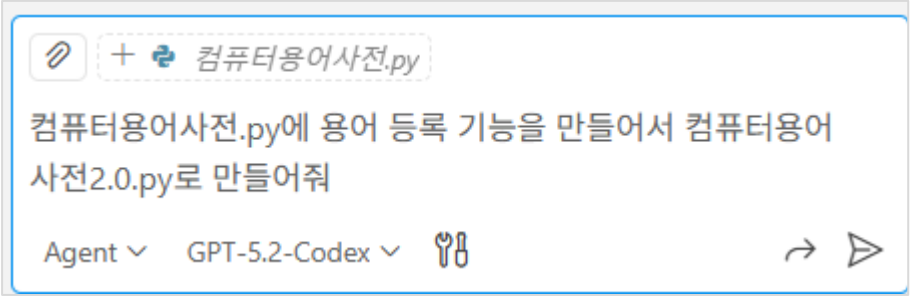
컴퓨터 용어사전 앱

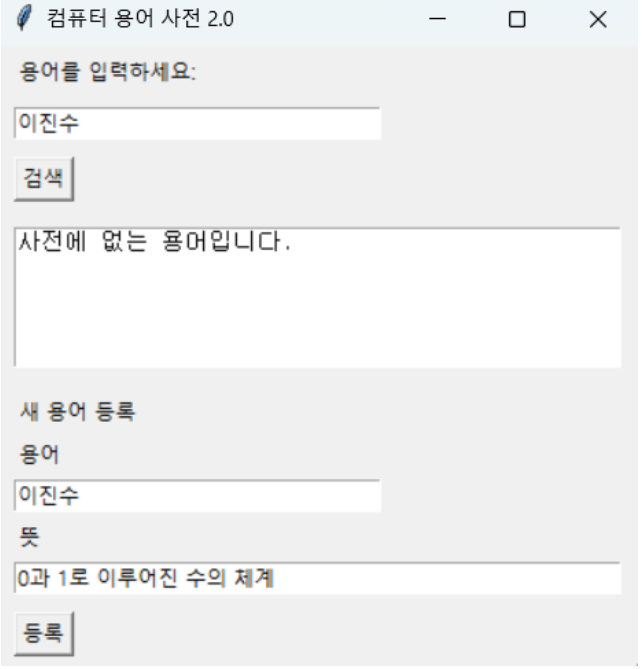
- 컴퓨터 용어사전.py

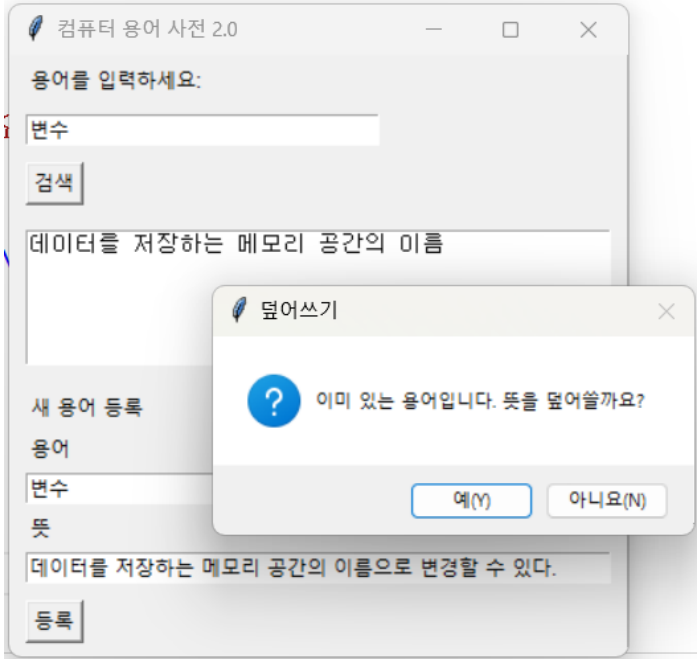
```
root = Tk()
root.title("컴퓨터 용어 사전")

Label(root, text="용어를 입력하세요:") \
    .grid(row=0, column=0, sticky=W, padx=10, pady=5) # 라벨 추가

entry = Entry(root, width=30)
entry.grid(row=1, column=0, sticky=W, padx=10, pady=5) # 입력창 추가

Button(root, text="검색", command=search) \
    .grid(row=2, column=0, sticky=W, padx=10, pady=5) # 버튼 추가

output = Text(root, width=50, height=10)
output.grid(row=3, column=0, sticky=W, padx=10, pady=10) # 출력창 추가

root.mainloop()
```

컴퓨터 용어사전 앱

- **코파일럿 채팅 입력**

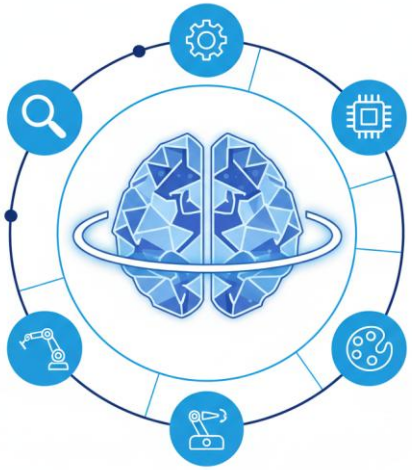






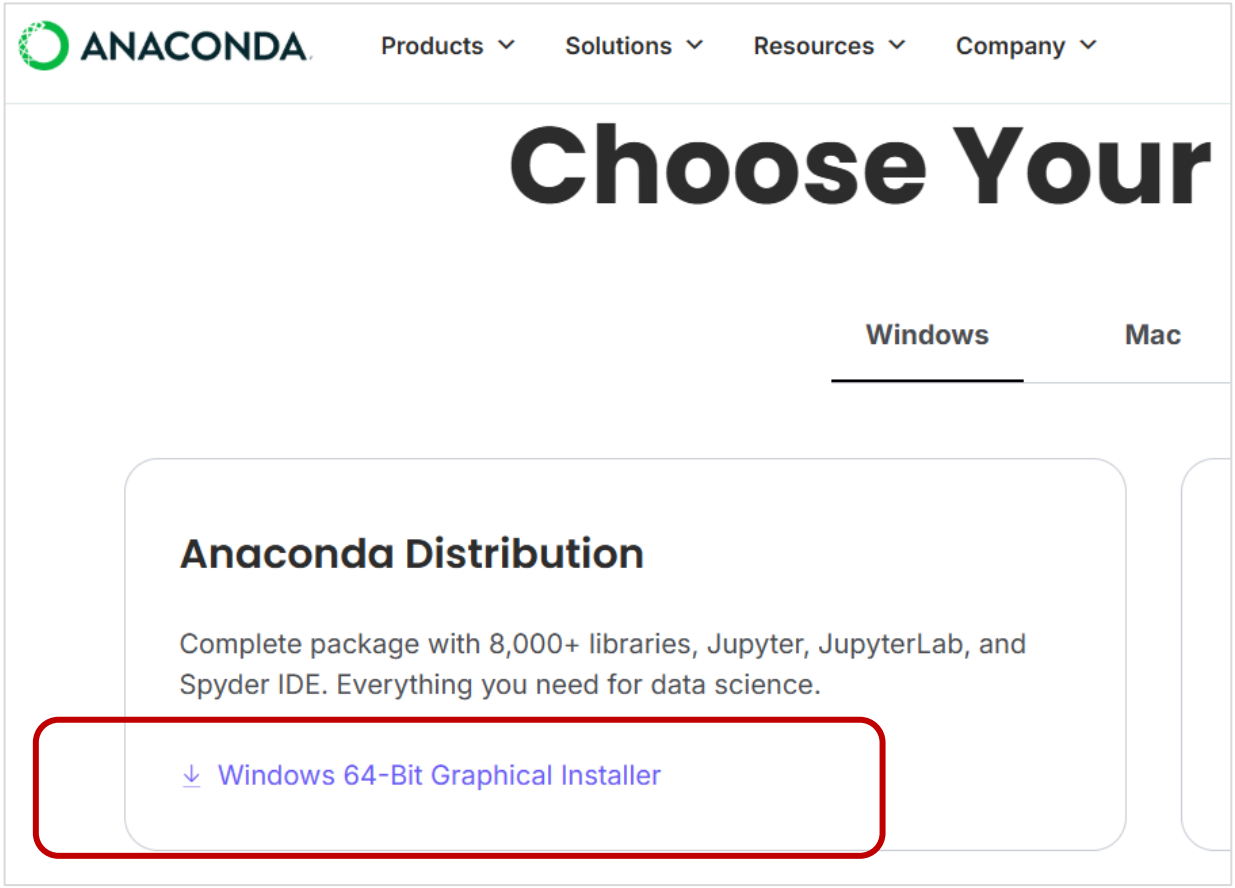
데이터 분석 및 시각화

pandas / matplotlib



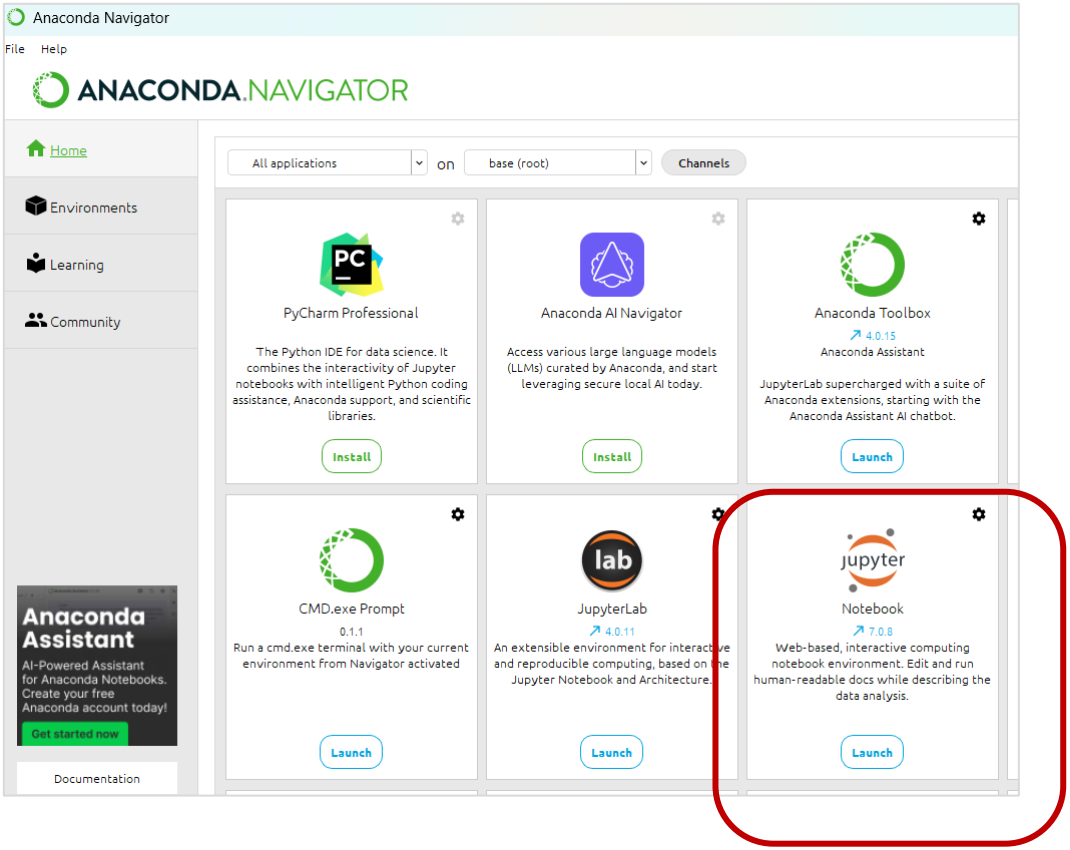
아나콘다(Anaconda) 플랫폼

❖ **Anaconda Download**



아나콘다(Anaconda) 플랫폼

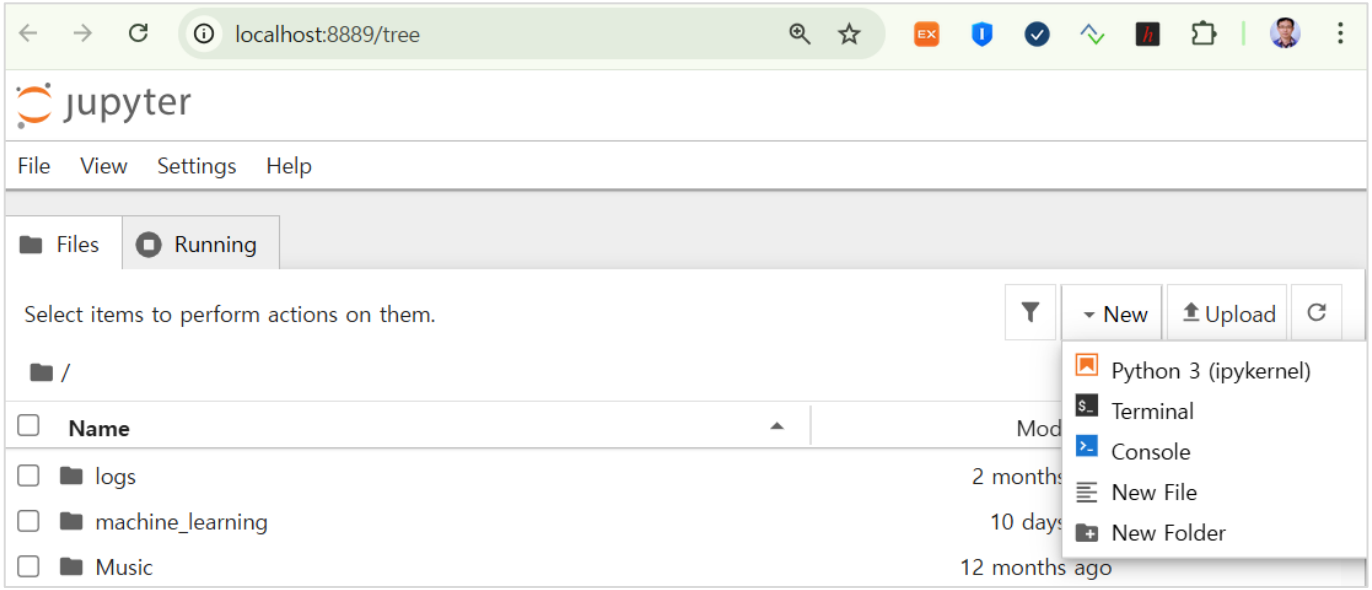
❖ 주피터 노트북(Jupyter Notebook) IDE 실행

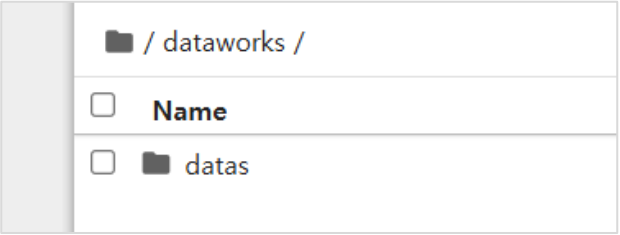


주피터 노트북(Jupyter Notebook)

❖ 작업 디렉터리 생성

New > New Folder > dataworks 폴더 생성 > 하위에 datas 폴더 생성

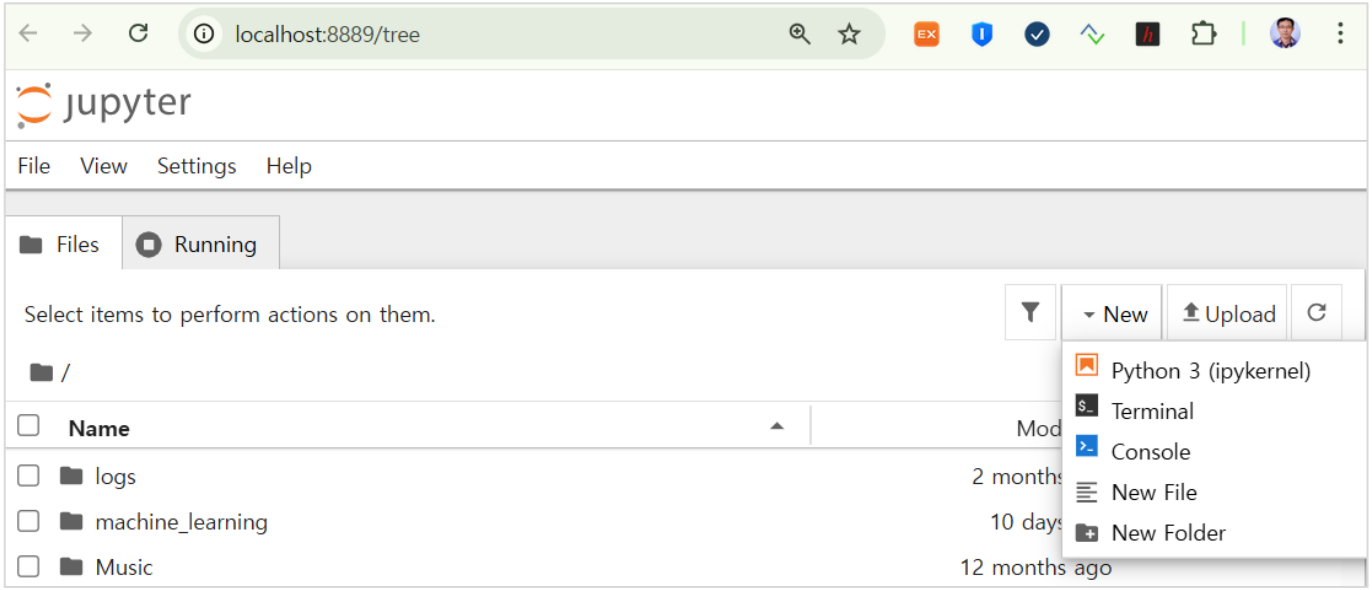


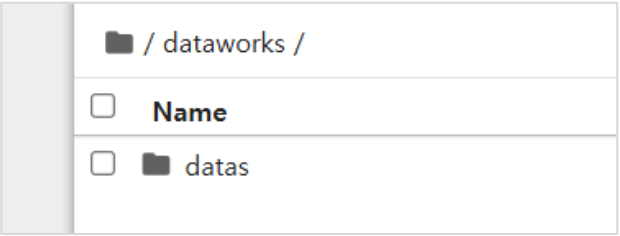


주피터 노트북(Jupyter Notebook)

❖ 작업 디렉터리 생성

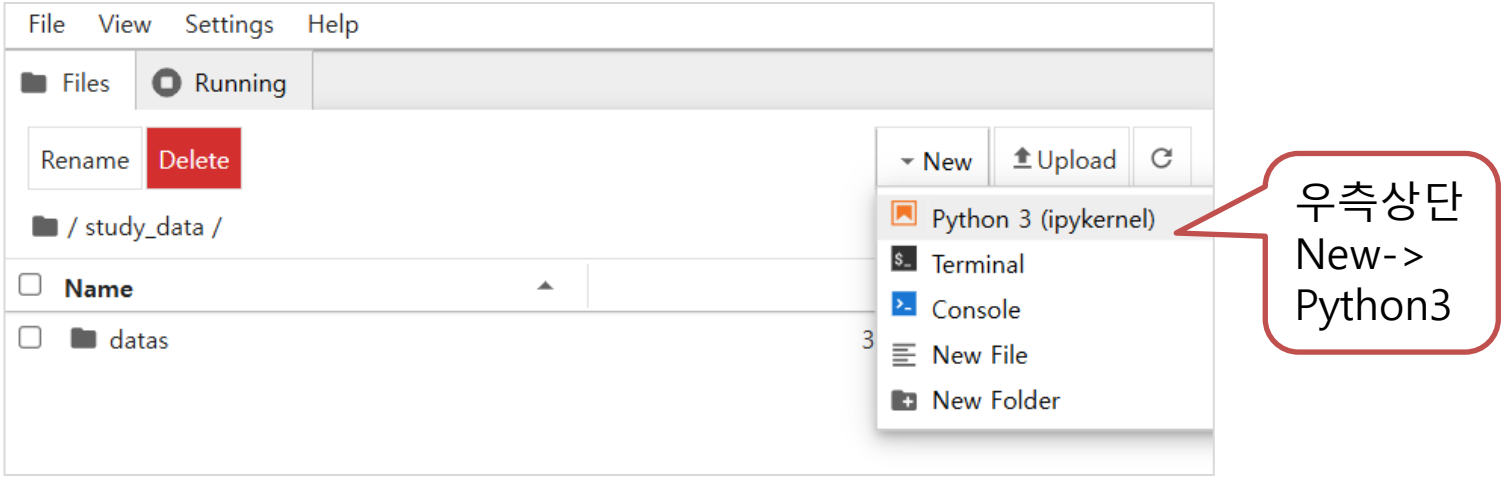
New > New Folder > dataworks 폴더 생성 > 하위에 datas 폴더 생성

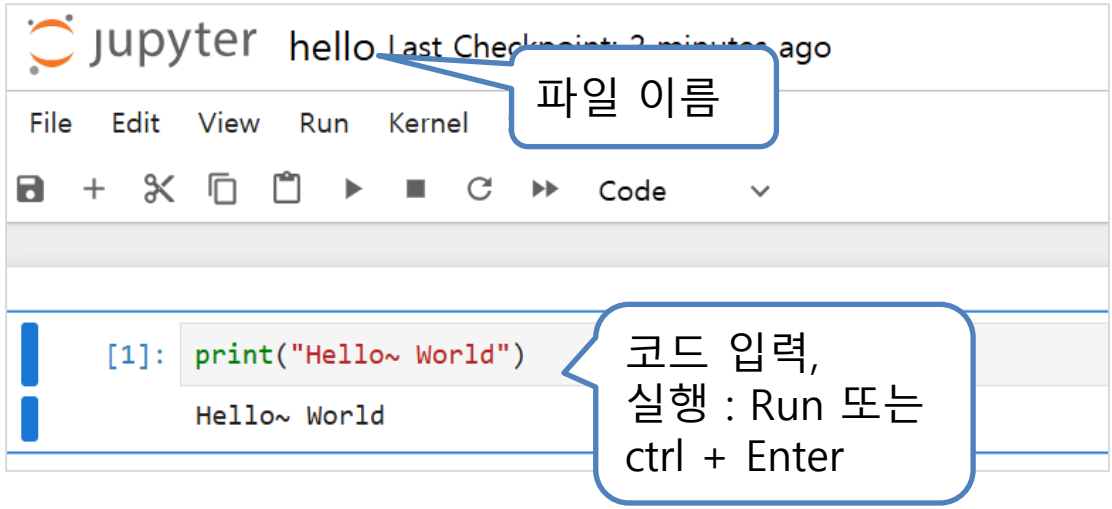




주피터 노트북(Jupyter Notebook)

❖ 파일 생성(.ipynb)





주피터 노트북(Jupyter Notebook)

❖ 코드 입, 출력

```
[1]: print("Hello~ World")

     Hello~ World

[2]: print("안녕~ 세계")

     안녕~ 세계

[7]: 10 + 20
     10 - 20

[7]: -10
```

```
[6]: n1 = 20
     n2 = 10

     print(n1 + n2)
     print(n1 - n2)
     print(n1 * n2)
     print(n1 / n2)
     print(n1 % n2)

     30
     10
     200
     2.0
     0
```

```
[10]: carts = ["계란", "라면", "콩나물"]
      print(carts)

      for cart in carts:
          print(cart)

      ['계란', '라면', '콩나물']
      계란
      라면
      콩나물
```

주피터 노트북(Jupyter Notebook)

❖ 세팅(Settings)

- 자동 완성 기능

  Home > Settings > Settings Editor > Enable autocompletion

- 줄번호, 글꼴 등

  Home > Settings > Notebook

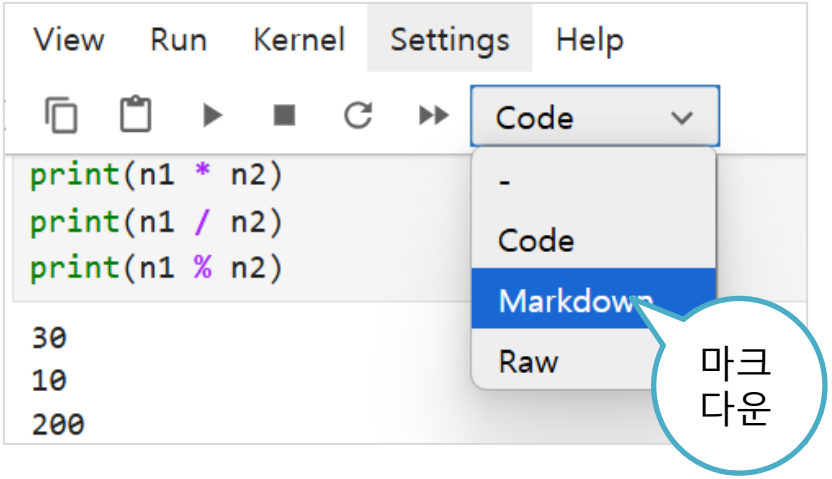
  - line numbers
  - Font Size : 20

주피터 노트북(Jupyter Notebook)

❖ Markdown

Markdown은 *설명·정리·문서화*를 담당하는 아주 중요한 기능이에요. 코드 옆에 **글, 표, 수식, 이미지**를 깔끔하게 적을 수 있어서 "노트 + 코드" 느낌으로 쓰기 딱 좋아요.

- 기호 종류
  - 제목: #의 개수가 클수록 작은 제목(## 텍스트)
  - 목록 – '*' 사용



주피터 노트북(Jupyter Notebook)

❖ Markdown 편집 및 실행 화면

```
### 함수 만들기
* 기능 - 절대값 계산
* 설명 - 음수는 양수로 양수는 양수로 반환
```

```
def my_abs(x):
    if x < 0:
        return -x
    else:
        return x

print(my_abs(-3))
```

```
3
```

<편집 화면>

Ctrl + Enter

함수 만들기

- 기능 - 절대값 계산
- 설명 - 음수는 양수로 양수는 양수로 반환

```
def my_abs(x):
    if x < 0:
        return -x
    else:
        return x

print(my_abs(-3))
```

```
3
```

<실행 화면>

데이터 분석

- **데이터 분석용 주요 라이브러리**

+ 데이터 분석을 위한 주요 라이브러리를 알려줘

라이브러리	주요 역할	핵심 기능
pandas	데이터 분석 핵심	DataFrame, CSV/Excel 처리, 필터링, 그룹화
numpy	수치계산 기반	다차원 배열, 수학/통계 연산, 고속 연산
matplotlib	데이터 시각화	선/막대/산점도 그래프

데이터 분석

- **판다스**
  - 데이터를 수집, 정리하는 데 최적화된 도구로 **표 형태(행·열)의 데이터**를 빠르고 편리하게 처리하기 위한 라이브러리입니다.
  - 가장 인기있는 프로그래밍 언어인 **파이썬(python)**을 기반으로 한다.
  - 시리즈(Series)와 데이터프레임(DataFrame)이라는 구조화된 데이터 형식을 제공한다.
  - 판다스 설치 - **아나콘다 플랫폼**에는 이미 설치되어 있습니다.

    (pip install pandas)

데이터 분석

- **판다스 자료구조**
  - **시리즈(Series)** – 데이터가 순차적으로 나열된 1차원 배열의 형태를 가지며, 인덱스(Index)는 데이터 값(value)과 일대일 대응이 된다.
  - **데이터프레임(DataFrame)** – 행(Row)과 열(Column)로 만들어지는 2차원 배열 구조로 표(Table) 형태이다.

	제품명
0	키보드
1	마우스
2	USB 메모리

인덱스 (index)

[시리즈]

	제품명	가격
0	키보드	20,000
1	마우스	32,000
2	USB 메모리	10,000

칼럼 (column)

[데이터프레임]

데이터 분석

- **Series(시리즈)**

import pandas as pd(별칭)

pd.Series(데이터, 인덱스)

속성	기능
index	- 시리즈의 인덱스 (생략하면 기본 0, 1, 2)
values	- 시리즈의 값(요소)
shape	- 시리즈의 크기 (요소의수,)

- **Series(시리즈)**

시리즈(Series) 만들기

```python
import pandas as pd

data = ['키보드', '마우스', 'USB 메모리']

s1 = pd.Series(data, index=[1, 2, 3])
print(s1)
print(type(s1))

# Series의 속성
print(s1.index)
print(s1.values)
print(s1.shape)

# 요소 선택 - 리스트 처럼 순서가 아닌 라벨 인덱스
print(s1[1])
print(s1[2])
```

```
1        키보드
2        마우스
3    USB 메모리
dtype: object
<class 'pandas.core.series.Series'>
Index([1, 2, 3], dtype='int64')
['키보드' '마우스' 'USB 메모리']
(3,)
키보드
마우스
```

데이터 분석

- **DataFrame(데이터프레임)**

```
import pandas as pd(별칭)

pd.DataFrame(데이터, 인덱스)
```

속성	기능
index	- 데이터프레임의 인덱스 (생략하면 기본 0, 1, 2)
columns	- 데이터프레임의 칼럼
shape	- 데이터프레임의 크기 (행, 열)

데이터 분석

칼럼(Column) 다루기

칼럼 추가 및 칼럼명 변경

```python
import pandas as pd

df = pd.DataFrame({
    "name": ['키보드', '마우스', 'USB 메모리'],
    "price": [20000, 32000, 11000]
})

# 칼럼 추가
df['spec'] = ['유선', '무선', '128GB']

df['할인가'] = df['price'] * 0.8
print(df)
print("----------------------------------------")

# 수정후 반드시 df에 다시 저장.
df = df.rename(columns={"name": "품명", "price":"가격", "spec":"제품상세"})
print(df)
```

```
      name  price   spec      할인가
0     키보드  20000    유선  16000.0
1     마우스  32000    무선  25600.0
2  USB 메모리  11000  128GB   8800.0
----------------------------------------
       품명     가격  제품상세      할인가
0     키보드  20000    유선  16000.0
1     마우스  32000    무선  25600.0
2  USB 메모리  11000  128GB   8800.0
```

데이터 분석

- **데이터 프레임을 CSV 파일로 변환**

CSV(Comma Seperated Value) 파일: 쉼표(,)로 데이터를 구분하여 저장하는 텍스트 파일 형식이다. 주로 표형태의 데이터를 저장하는 데 사용하며 엑셀에서 쉽고 열고 편집할 수 있음

☞ CSV 파일로 변환

df.to_csv("computer.csv", index=False)

☞ CSV 파일 읽기

pd.read_csv("computer.csv")

데이터 분석

- **데이터 프레임을 CSV 파일로 변환**

CSV 파일이란?

쉼표(,)로 데이터를 구분하여 저장하는 텍스트 파일 형식이다.

엑셀에서 .csv 형식으로 저장하고 불러올 수 있다.

```python
import pandas as pd

# csv 파일 만들기 - index 제외
df.to_csv("datas/computer.csv", index=False)

# csv 파일 읽기
print(pd.read_csv("datas/computer.csv"))
```

```
        품명     가격   제품상세      할인가
0      키보드  20000     유선  16000.0
1      마우스  32000     무선  25600.0
2  USB 메모리  11000  128GB   8800.0
```

/ study_data / datas /

Name
computer.csv

데이터 관리

- **행과 열 선택**

속성	범위	예시
loc[인덱스명, 칼럼명] (location)	끝 인덱스 포함	인덱스 - 라벨인덱스
iloc[인덱스번호, 칼럼번호] (integer location)	끝 인덱스 - 1	인덱스 – 순서(0부터 시작)

데이터 관리

- **행/열 선택 : loc[행이름, 열이름] 사용**

```python
# csv 파일 읽어서 df 객체에 저장
df = pd.read_csv("datas/computer.csv")

# 행 선택 - loc[행(행이름)]
print(df.loc[0:1])
print("========================================")

# 특정 요소 선택 - loc[행이름, 열(칼럼명)]
print(df.loc[0, "품명"])  #키보드
print(df.loc[1, "가격"])  #32000
print(df.loc[0:1, "품명"])
print(df.loc[0:1, ["품명", "가격"]])
```

```
    품명     가격 제품상세      할인가
0  키보드  20000   유선  16000.0
1  마우스  32000   무선  25600.0
========================================
키보드
32000
0    키보드
1    마우스
Name: 품명, dtype: object
    품명     가격
0  키보드  20000
1  마우스  32000
```

데이터 관리

- **행/열 선택 : iloc[행번호, 열번호] 사용**

```
# 행 선택 - iloc[행(인덱스번호)]
print(df.iloc[0:2]) # 끝인덱스-1
print("========================================")

# 특정 요소 선택 - loc[행(인덱스번호), 열(인덱스번호)]
print(df.iloc[0, 0]) #키보드
print(df.iloc[1, 1]) #32000
print(df.iloc[0:2, 0])
print(df.iloc[0:2, 0:2])
```

```
    품명     가격 제품상세      할인가
0  키보드  20000   유선  16000.0
1  마우스  32000   무선  25600.0
========================================
키보드
32000
0    키보드
1    마우스
Name: 품명, dtype: object
    품명     가격
0  키보드  20000
1  마우스  32000
```

데이터 관리

- 데이터 프레임 - 요소값 변경(업데이트)

df.loc[행이름, 열이름] = “변경할 값 "

df.iloc[행번호, 열번호] = “변경할 값 "

- 데이터 프레임 - 요소값 추가

df[열] = “추가할 칼럼“ => 열 추가

df.loc[행] = “추가할 값“ => 행 추가

데이터 관리

- **데이터 프레임 - 행/열 요소 변경 및 추가**

```python
# 요소값 변경 및 추가
df2 = df.copy() #원본 유지 복사
df2

# 품명 '마우스'를 '무선마우스'로 변경
df2.loc[1, "품명"] = "무선마우스"
# 무선 마우스의 가격을 35000으로 변경
df2.loc[1, "가격"] = 35000
# 무선 마우스의 할인가 수정
df2.loc[1, "할인가"] = df2.loc[1, "가격"] * 0.8
df2

# 행 추가
df2.loc[3] = ["키보드", 40000, "무선", 0.0]
df2.loc[3, "할인가"] = df2.loc[3, "가격"] * 0.8
df2
```

	품명	가격	제품상세	할인가
0	키보드	20000	유선	16000.0
1	무선마우스	35000	무선	28000.0
2	USB 메모리	11000	128GB	8800.0
3	키보드	40000	무선	32000.0

데이터 관리

- 데이터 프레임 - 행/열 삭제

df.drop(행, axis=0) => 행 삭제

df.drop(열, axis=1) => 열 삭제

```
# 행 삭제 - drop(행번호, axis=0)
# 열 삭제 - drop(칼럼명, axis=1)
df3 = df2.copy()
df3

# 3행 삭제
df3 = df3.drop(3, axis=0)
df3

# 열 삭제
# df3 = df3.drop("할인가", axis=1)
df3 = df3.drop(["제품상세", "할인가"], axis=1)
df3
```

	품명	가격
0	키보드	20000
1	무선마우스	35000
2	USB 메모리	11000

데이터 탐색

- **데이터 프레임을 CSV 파일로 변환**

```
# 데이터프레임 만들기
food = pd.DataFrame({
    "메뉴" : ["김밥", "비빔밥", "자장면", "우동", "김밥", "자장면", "김밥"],
    "가격" : [3000, 7000, 5500, 5500, 3500, 6000, 4000],
    "분류" : ["한식", "한식", "중식", "일식", "한식", "중식", "한식"]
})
# print(food)

# csv 파일로 변환
food.to_csv("food.csv", index=False)

# csv 파일 읽어오기
df = pd.read_csv("food.csv")
print(df)
```

데이터 탐색

- **데이터 탐색을 위한 주요 함수**

```
# 데이터 출력 - head(), tail() -> 기본 5개
print(df.head())
print(df.tail())

# 데이터 정보 - info()
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   메뉴      7 non-null      object
 1   가격      7 non-null      int64
 2   분류      7 non-null      object
dtypes: int64(1), object(2)
memory usage: 300.0+ bytes
```

데이터 탐색

- **데이터 탐색을 위한 주요 함수**

```python
# 고유한 값의 개수 - value_counts()
print(df["메뉴"].value_counts())
print(df["분류"].value_counts())

# 통계 요약 - describe()
# print(df.describe())
print(df.describe(include='number'))
print(df.describe(include='object'))
```

```
메뉴
김밥      3
자장면     2
비빔밥     1
우동      1
Name: count, dtype: int64
분류
한식     4
중식     2
일식     1
Name: count, dtype: int64
```

	가격
count	7.000000
mean	4928.571429
std	1455.694892
min	3000.000000
25%	3750.000000
50%	5500.000000
75%	5750.000000
max	7000.000000

데이터 탐색

- **조건 검색(필터링)**

비교 연산 기호	설명
>, >=	크다, 크거나 같다.
<, <=	작다, 작거나 같다
==	두 항이 같다.
!=	두 항이 같지 않다.

논리 연산	연산 기호	설명
교집합(AND)	&	두 조건이 모두 참일때 True 반환
합집합(OR)	|	두 조건중 하나 이상이 참일때 True 반환
부정(NOT)	~	조건이 참이면 False, 거짓이면 True를 반환

데이터 탐색

- **데이터 탐색 – 조건 검색(필터링)**

```
# 조건 검색(필터링)
# 메뉴가 '자장면'인 음식의 정보
result1 = (df["메뉴"] == '자장면')
print(df[result1])

# 메뉴가 '김밥'이 아닌 음식의 정보
result2 = (df["메뉴"] != '김밥')
# result2 = ~(df["메뉴"] == '김밥')
print(df[result2])

# 가격이 4000원 미만인 음식의 정보
result3 = (df["가격"] < 4000)
print(df[result3])

# 메뉴가 '자장면'이고, 가격이 6000원 이상인 음식의 정보
# result4 = (df["메뉴"] == '자장면') & (df["가격"] >= 6000)
result4 = (df["메뉴"] == '자장면') | (df["가격"] >= 6000)
print(df[result4])
```

데이터 탐색

● 데이터 탐색 - 정렬

정렬 분류	함수	파라미터
칼럼 기준	sort_values(칼럼, 파라미터)	오름차순: ascending=True 내림차순: ascending=False
인덱스 기준	sort_index(파라미터)	

```
     메뉴    가격   분류
1   비빔밥  7000   한식
5   자장면  6000   중식
2   자장면  5500   중식
3    우동   5500   일식
6    김밥   4000   한식
4    김밥   3500   한식
0    김밥   3000   한식
```

```
     메뉴    가격   분류
6    김밥   4000   한식
4    김밥   3500   한식
0    김밥   3000   한식
1   비빔밥  7000   한식
3    우동   5500   일식
5   자장면  6000   중식
2   자장면  5500   중식
```

데이터 탐색

- **데이터 탐색 - 정렬**

```
df = pd.read_csv("datas/food.csv")
# print(df)
# print("========================================")

# 가격을 내림차순으로 정렬하기
df.sort_values("가격", ascending=False, inplace=True)
print(df)
print("========================================")

# 메뉴를 오름차순으로 정렬, 메뉴가 같으면 가격을 내림차순 정렬
df.sort_values(["메뉴", "가격"], ascending=[True, False], inplace=True)
print(df)
print("========================================")

# 인덱스 기준 정렬(초기화)
df.sort_index(ascending=True)
print(df)
```

데이터 탐색

- **판다스의 통계, 수학 , 그룹핑 함수**

함수	기능
count() / sum() / mean()	개수 / 합계 / 평균
var() / std()	분산 / 표준편차
max() / min()	최대값 / 최소값
idxmax() / idxmin()	최대값 위치 / 최소값 위치
quantile(.25) / quantile(.75)	1사분위수 / 3사분위수
mode()	빈도값
round(수, 자리수)	반올림
groupby(칼럼명)	그룹화

데이터 탐색

- 판다스의 통계, 수학 함수

```
print("개수:", df["가격"].count())
print("합계:", df["가격"].sum())
print("평균:", round(df["가격"].mean(), 2))
print("분산:", round(df["가격"].var(), 2))
print("표준편차:", round(df["가격"].std(), 2))
print("최대값:", df["가격"].max())
print("최소값:", df["가격"].min())
print("최대값의 위치:", df["가격"].idxmax())
print("최소값의 위치:", df["가격"].idxmin())
```

데이터 탐색

- **판다스의 통계, 수학 함수**

```
# 사분위수 - quantile()
print("1사분위수:", df["가격"].quantile(.25))
print("2사분위수:", df["가격"].quantile(.50))
print("3사분위수:", df["가격"].quantile(.75))
print("4사분위수:", df["가격"].quantile(1.0))

# 조건 - 1사분위수 보다 작은 가격을 출력
result = df["가격"] < df["가격"].quantile(.25)
print(df[result])

# 최빈값 - mode()
print("최빈값:", df["가격"].mode()[0])
```

데이터 탐색

- **그룹핑 - groupby()**

```python
import pandas as pd

# "수량" 칼럼 추가
df["수량"] = [10, 10, 15, 5, 7, 7, 8]
df

# csv 파일로 변환
df.to_csv("./datas/food2.csv", index=False)

# csv 파일 읽기
food = pd.read_csv("./datas/food2.csv")
food
```

```
메뉴,가격,분류,수량
김밥,3000,한식,10
비빔밥,7000,한식,10
자장면,5500,중식,15
우동,5500,일식,5
김밥,3500,한식,7
자장면,6000,중식,7
김밥,4000,한식,8
```

데이터 탐색

- **그룹핑 - groupby()**

```
food = pd.read_csv("food2.csv")
# print(food)

# 그룹화 - groupby()
df["분류"].value_counts()

food.groupby("메뉴").mean(numeric_only=True)
food.groupby("분류").mean(numeric_only=True) #모든 수치 칼럼 평균

food.groupby("분류")["가격"].mean()  #분류별 가격의 평균
food.groupby("분류")["수량"].mean()  #분류별 수량의 평균
```

데이터 전처리

- **결측치 삭제 및 대체**

함수	설명
drop(행, axis=0) drop(열, axis=1)	NaN이 포함된 행 삭제 NaN이 포함된 열 삭제
dropna()	NaN이 포함된 모든 행 삭제
fillna()	- 수치형: 평균(mean()), 중간값(median()) - 문자형: 빈도값(mode())
isnull() isnull().sum()	- 결측치 확인(True / False로 반환) - 결측치 개수(True(1))

데이터 전처리

- **결측치 대체**

```
data = {
    "메뉴": ["김밥", "비빔밥", "자장면", "우동", None, "자장면", "김밥"],
    "가격": [3000, 7000, 5500, None, 3500, 6000, 4000],
    "분류": ["한식", "한식", "중식", "일식", "한식", "중식", "한식"]
}

food = pd.DataFrame(data)
food
food.info()

# 결측치 확인 - isnull()
print(food.isnull())
print(food.isnull().sum()) #True(1) / False(0)
```

```
메뉴    1
가격    1
분류    0
```

데이터 전처리

- **결측치 대체**

```
# 수치형 - 중간값으로 대체
median = food["가격"].median()
median

# fillna()
food["가격"] = food["가격"].fillna(median)
food

# 문자형 - 최빈값
frequency = food["메뉴"].mode()[0]
frequency
food["메뉴"] = food["메뉴"].fillna(frequency)
food

print(food.isnull().sum())
```

데이터 시각화

- **matplotlib 라이브러리**

+ matplotlib 시각화 라이브러리에 대해 설명해줘

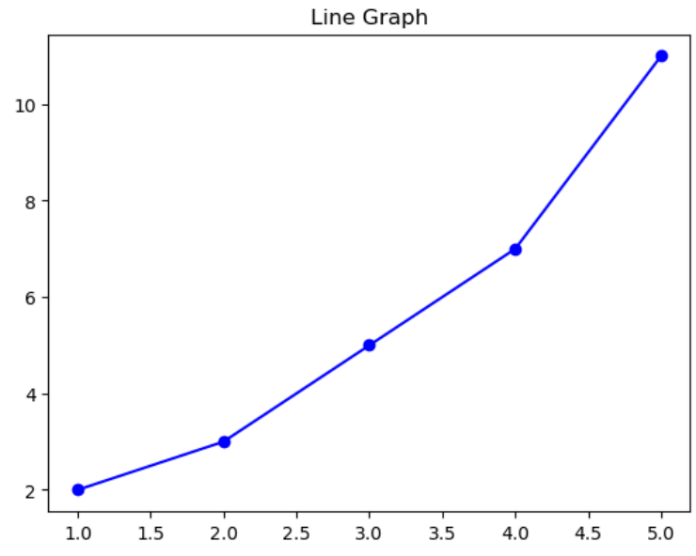
- 데이터 분석, 과학 시각화, 보고서 작성 등 다양한 분야에서 그래프를 그릴 때 사용됩니다.
- 주요 그래프 종류

종류	함수명	예시
선 그래프	plot()	plt.plot(x, y)
막대 그래프(수직/수직)	bar() / barh()	plt.bar(x, y) / plt.barh(x, y)
산점도	scatter()	plt.scatter(x, y)
히스토그램	hist()	plt.hist(data)
파이차트	pie()	plt.pie(data)

77

데이터 시각화

- 선 그래프



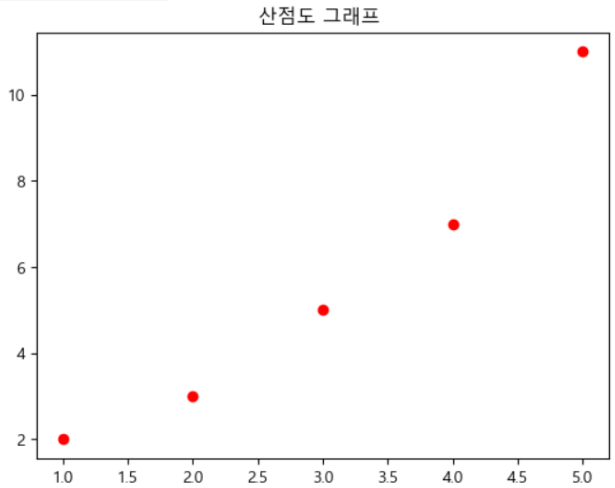

```python
import matplotlib.pyplot as plt

# 데이터 준비
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# 선 그래프 그리기
plt.plot(x, y, label='Prime numbers', color='blue', marker='o')
plt.title('Line Graph') #제목
plt.legend() #범례

# 그래프 출력
plt.show()
```


78

데이터 시각화

- 산점도 그래프

```
import matplotlib.pyplot as plt

# 데이터 준비
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]  #소수

plt.rc('font', family='Malgun Gothic')
plt.title('산점도 그래프 ')
plt.scatter(x, y, color='red')
plt.show()
```

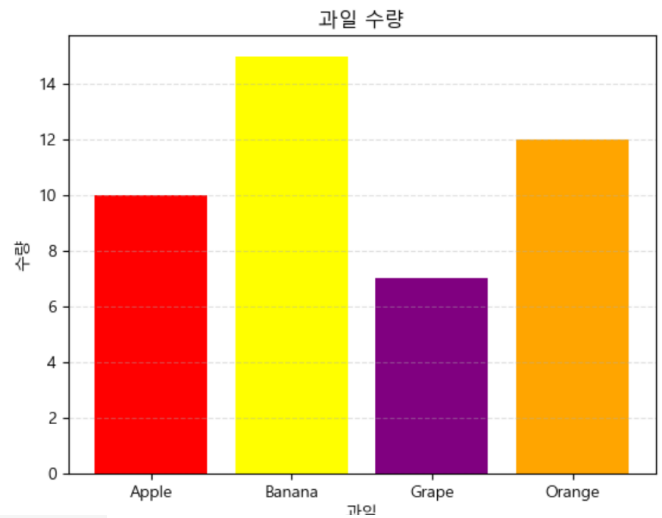


79

데이터 시각화

- 막대 그래프

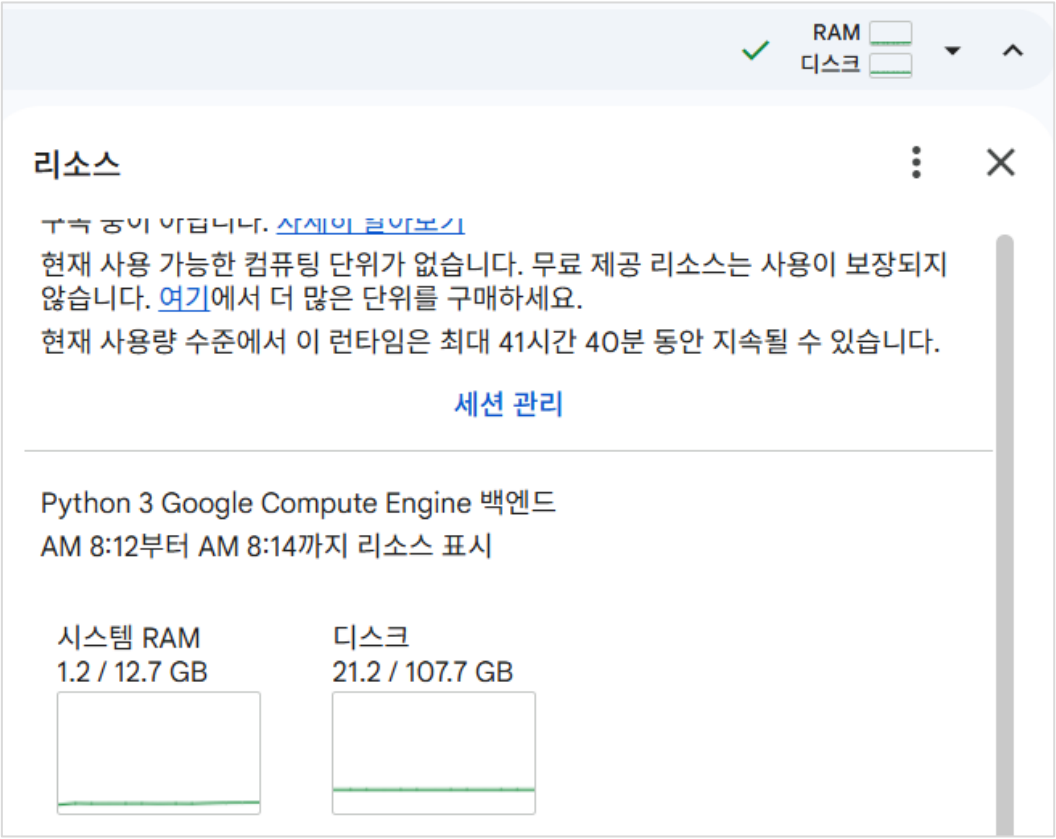
```python
import matplotlib.pyplot as plt

# 데이터 준비
fruits = ['Apple', 'Banana', 'Grape', 'Orange']
counts = [10, 15, 7, 12]
color = ['red', 'yellow', 'purple', 'Orange']

# 한글 깨짐 방지 - Batang, Gulim 사용
plt.rc('font', family="Malgun Gothic")

# 막대 그래프
plt.bar(fruits, counts, color=color)
plt.title('과일 수량')
plt.xlabel('과일') # x축 제목
plt.ylabel('수량') # y축 제목

# y축 grid 추가
# plt.grid()
plt.grid(axis='y', linestyle='--', alpha=0.3)
plt.show()
```



80

구글 코렙

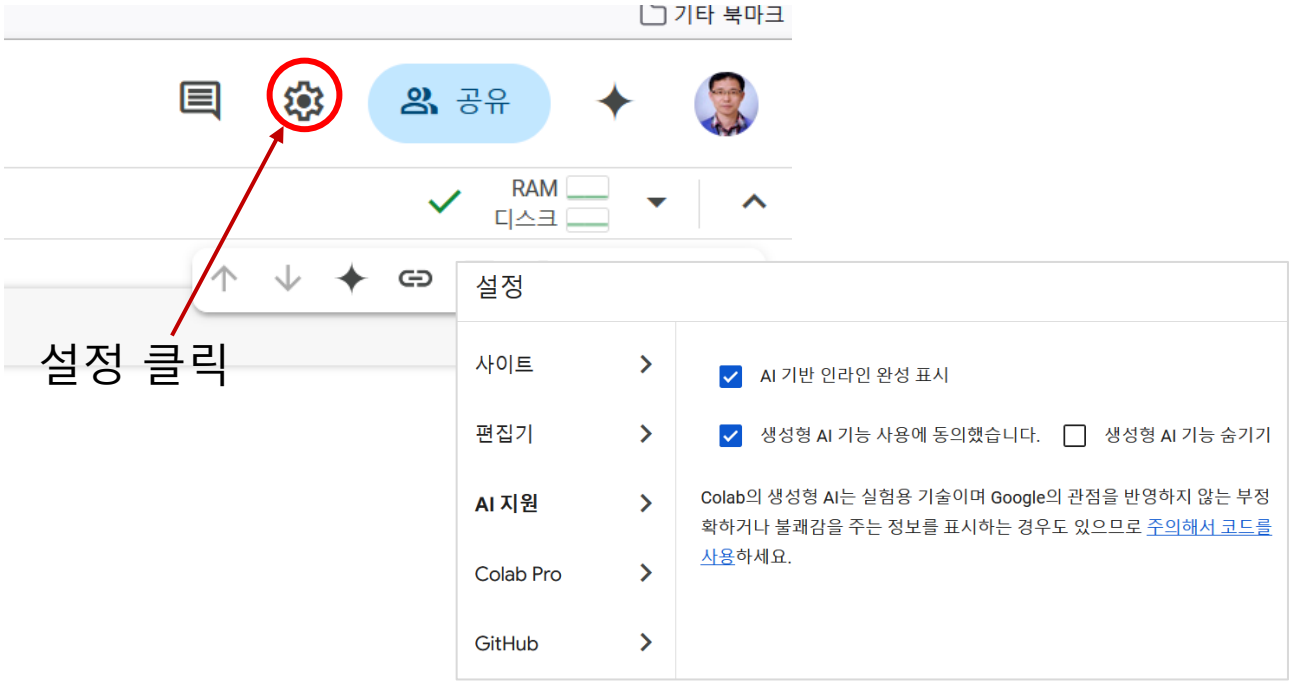
❖ 리소스 사용 – 클라우드 컴퓨터 사용



구글 코렙

❖ 구글 코랩(Colaboratory)

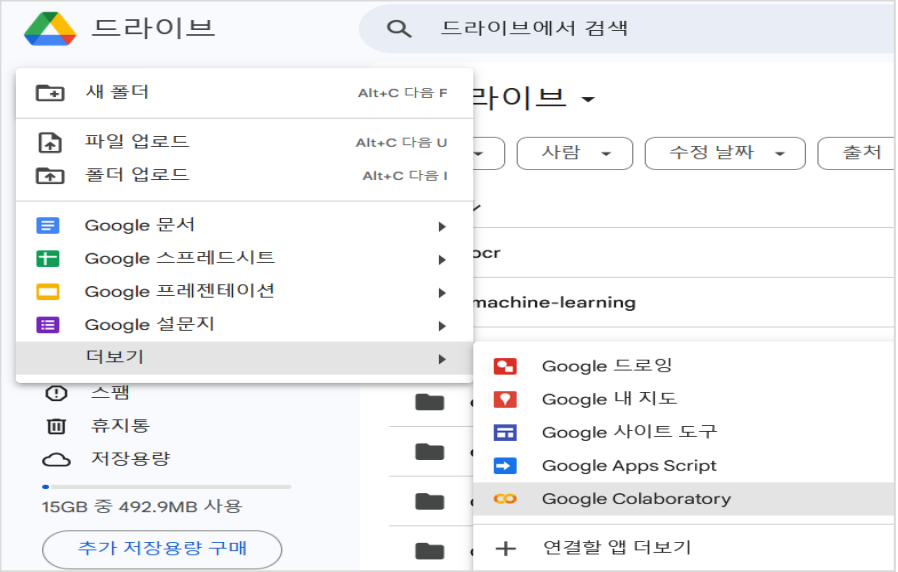
AI(Gemini) 지원 : 설정 > AI 지원 > 동의



구글 코렙

❖ 새 노트 만들기

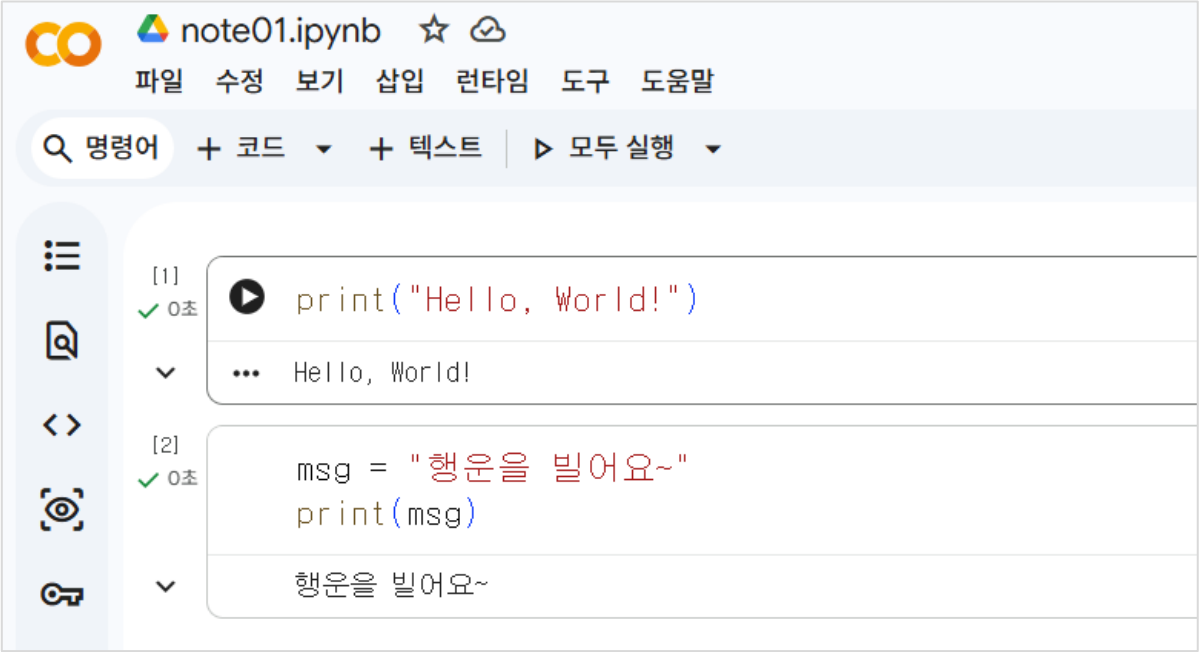
- 드라이브 > 홈
- + 신규 > 더보기 > Google Colaboratory



구글 코렙

❖ 새 노트 만들기

- 파일이름 > note01.ipynb



구글 코렙

❖ 새 노트 만들기

- 파일이름 > note01.ipynb

```python
import pandas as pd

df = pd.DataFrame({
    "name": ['마우스', '모니터'],
    "price": [20000, 80000]
})

df
```

	name	price
0	마우스	20000
1	모니터	80000

다음 단계: df 변수로 코드 생성 | New interactive sheet

구글 코렙

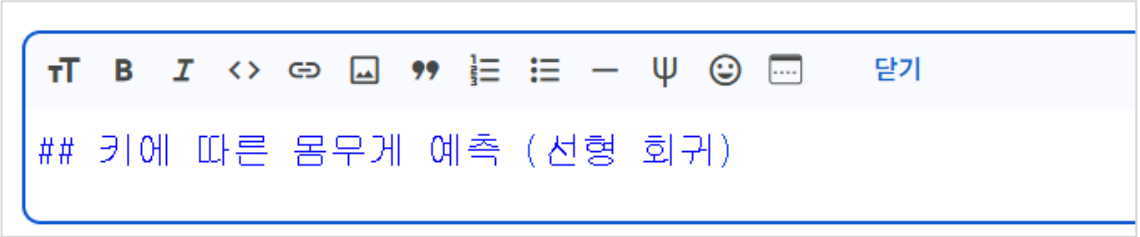
❖ 새 폴더 만들기

- 드라이브 > 내 드라이브
- + 신규 > 새폴더> 머신러닝

구글 코렙

❖ 텍스트 사용하기

- 파일 이름 : 선형회귀.ipynb
- 텍스트(버튼) > 문서의 제목을 만들때 사용



<편집화면>

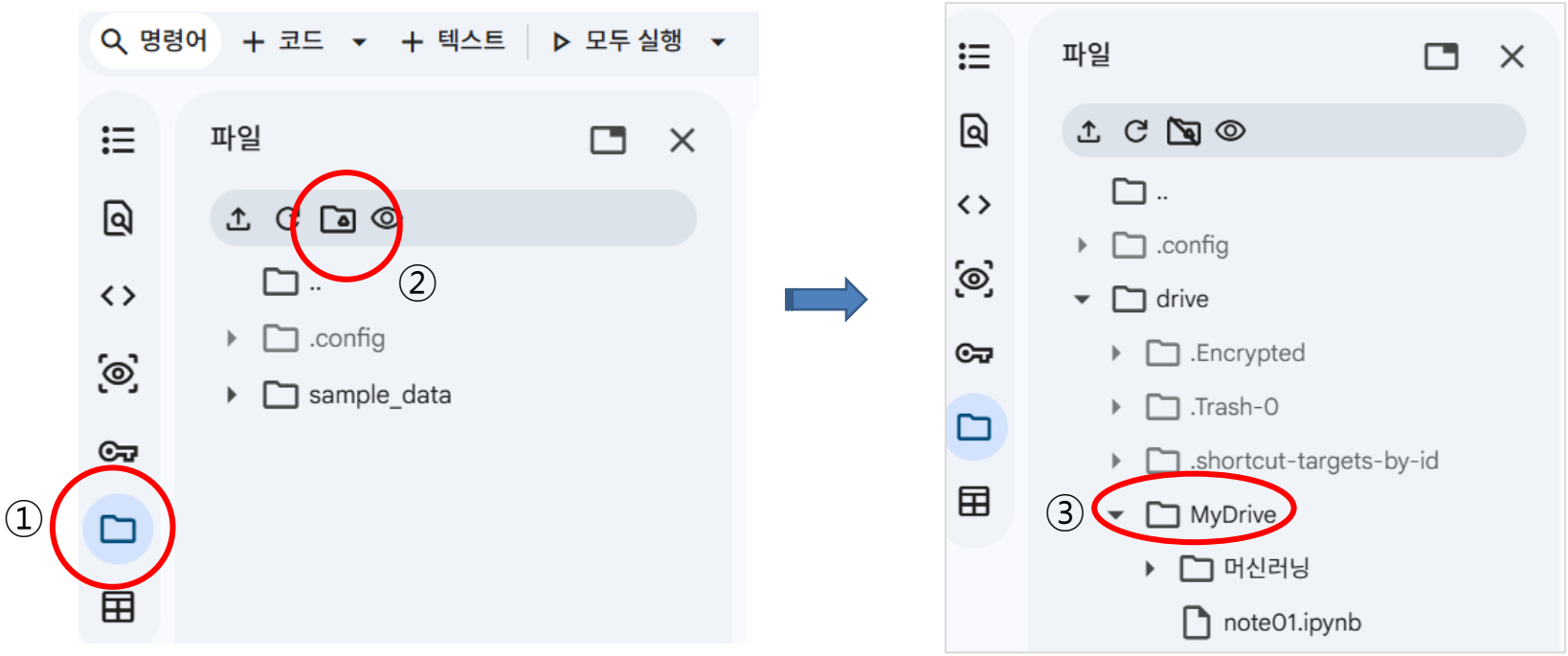
shift + enter

키에 따른 몸무게 예측 (선형 회귀)

<출력화면>

❖ 드라이브 마운트

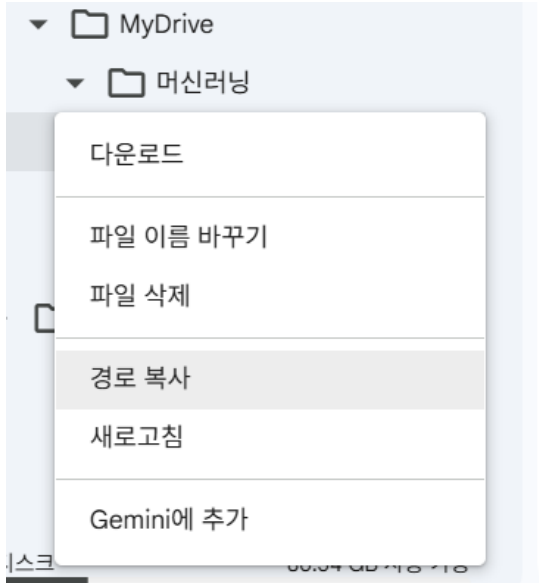
- 폴더 > 드라이브 마운트 > 드라이브 연결

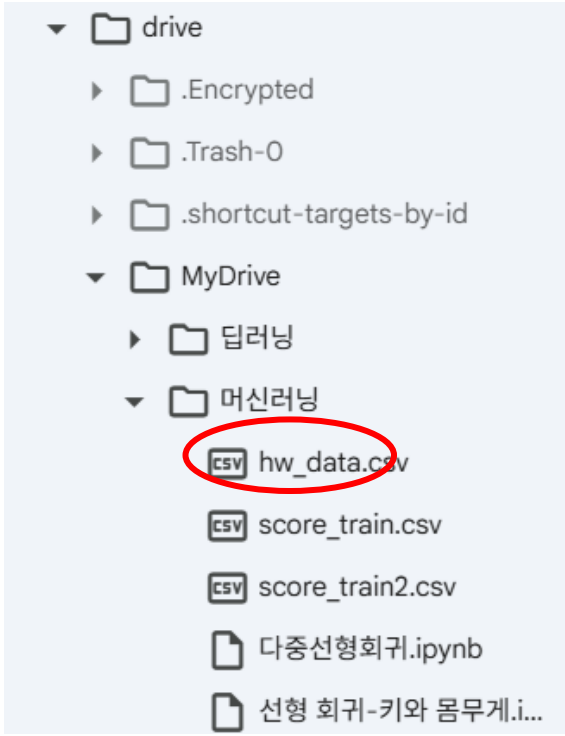


구글 코렙

❖ 훈련 데이터 업로드

- (...) 메뉴 > 업로드 > data_hw.csv





상관 분석 VS 회귀 분석

- **상관 분석 vs 회귀 분석**

상관 분석	회귀 분석
x와 y의 관련 정도를 확인	x가 y에 주는 영향을 추정하여 결과를 예측할 수 있음
산포도, 상관 계수	회귀식
광고비와 매출은 양의 상관관계가 있다.	광고비를 1% 늘리면 매출이 5% 증가할 것이다.

90

데이터 시각화

- **seaborn 라이브러리**
  - Python의 데이터 시각화 라이브러리로, 통계적 그래프를 쉽게 그릴 수 있도록 matplotlib을 기반으로 만들어졌다
  - pandas의 DataFrame과 잘 통합되어 있어, 데이터를 시각적으로 분석하고자 할 때 매우 유용하다

- **설치 방법**

  pip install seaborn

91

상관 분석 vs 회귀 분석

- **데이터 준비 및 탐색**

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('car_crashes')
df.head()

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51 entries, 0 to 50
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   total           51 non-null     float64
 1   speeding        51 non-null     float64
 2   alcohol         51 non-null     float64
 3   not_distracted  51 non-null     float64
 4   no_previous     51 non-null     float64
 5   ins_premium     51 non-null     float64
 6   ins_losses      51 non-null     float64
 7   abbrev          51 non-null     object
dtypes: float64(7), object(1)
memory usage: 3.3+ KB
```


92

상관 분석 vs 회귀 분석

- **데이터 준비 및 탐색**

변수	의미
total	10만 명당 총 사고 건수
speeding	과속 관련 사고 비율
alcohol	음주 관련 사고 비율
not_distracted	비산만 상태 사고 비율
no_previous	이전 사고 기록 없음 비율
ins_premium	보험료
ins_losses	보험 손실 비용
abbrev	주 약자 (문자형 → 상관분석 제외됨)

93

상관 분석 vs 회귀 분석

- **상관 계수 및 시각화**

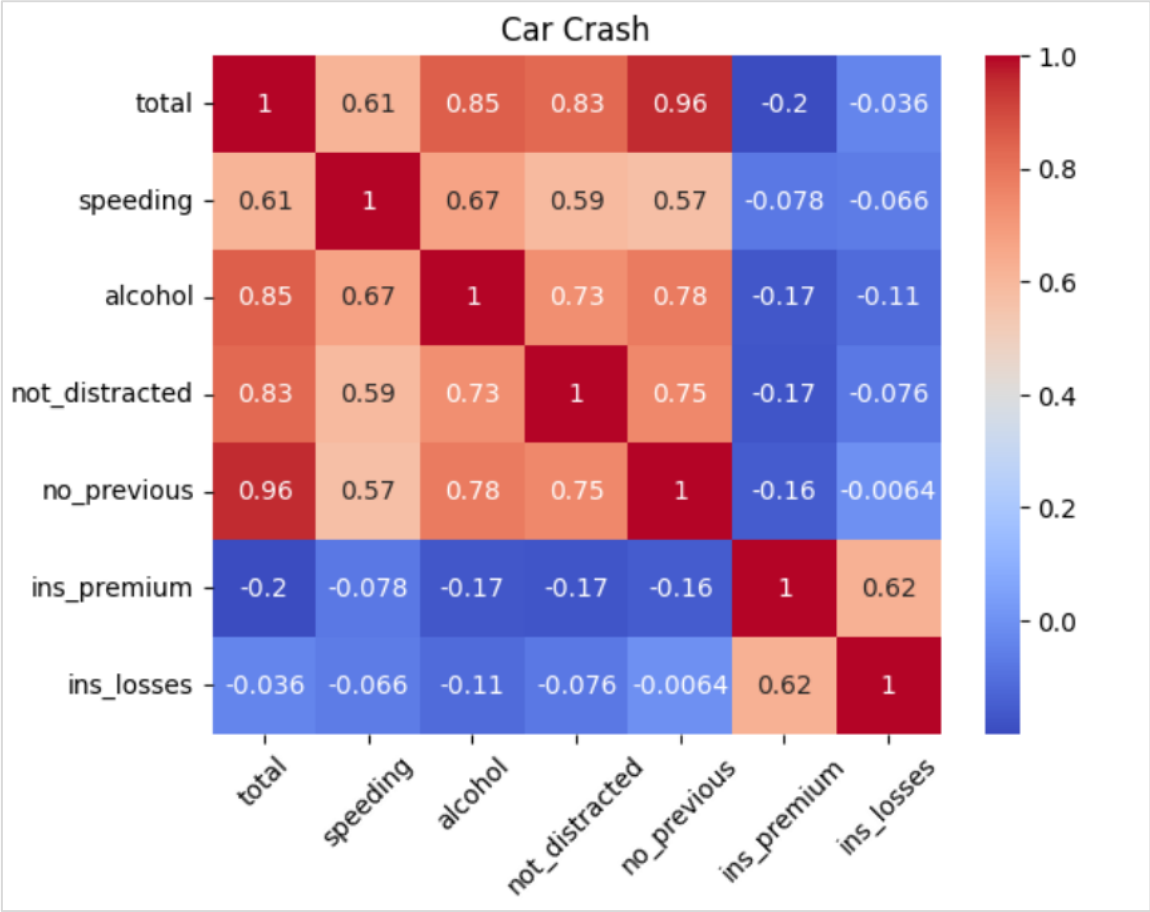
```
# 상관 계수
# corr = df.select_dtypes(include=['number']).corr()
corr = df.select_dtypes(exclude=['object']).corr()
corr

# 시각화 - 히트맵
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Car Crash')
plt.xticks(rotation=45)
```

```
(array([0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5]),
 [Text(0.5, 0, 'total'),
  Text(1.5, 0, 'speeding'),
  Text(2.5, 0, 'alcohol'),
  Text(3.5, 0, 'not_distracted'),
  Text(4.5, 0, 'no_previous'),
  Text(5.5, 0, 'ins_premium'),
  Text(6.5, 0, 'ins_losses')])
```


94

상관 분석 vs 회귀 분석

- 상관 계수 및 시각화



95

상관 분석 vs 회귀 분석

- **주요 상관 관계 분석**

✅ **① total ↔ speeding**

- **강한 양의 상관관계**
- 총 사고 건수가 많을수록 과속 관련 사고도 많음
- 📌 해석: 과속이 전체 사고 증가에 큰 영향을 미칠 가능성

✅ **② total ↔ alcohol**

- **양의 상관관계**
- 음주 사고 비율이 높은 주는 전체 사고율도 높은 경향
- 📌 해석: 음주 역시 사고의 주요 요인

✅ **③ speeding ↔ alcohol**

- 보통 **중간 정도의 양의 상관관계**
- 과속과 음주가 동시에 나타나는 경향

👉 즉, 위험 운전 행동들이 함께 발생하는 경향이 있음

✅ **④ ins_premium ↔ ins_losses**

- **매우 강한 양의 상관관계**
- 보험 손실이 높을수록 보험료도 높음
- 📌 매우 자연스러운 결과 (보험사는 손실 반영)

96